DM-Call: Decision-Model-Guided Function Calling with Cumulative Verification Feedback

Anonymous Author(s)

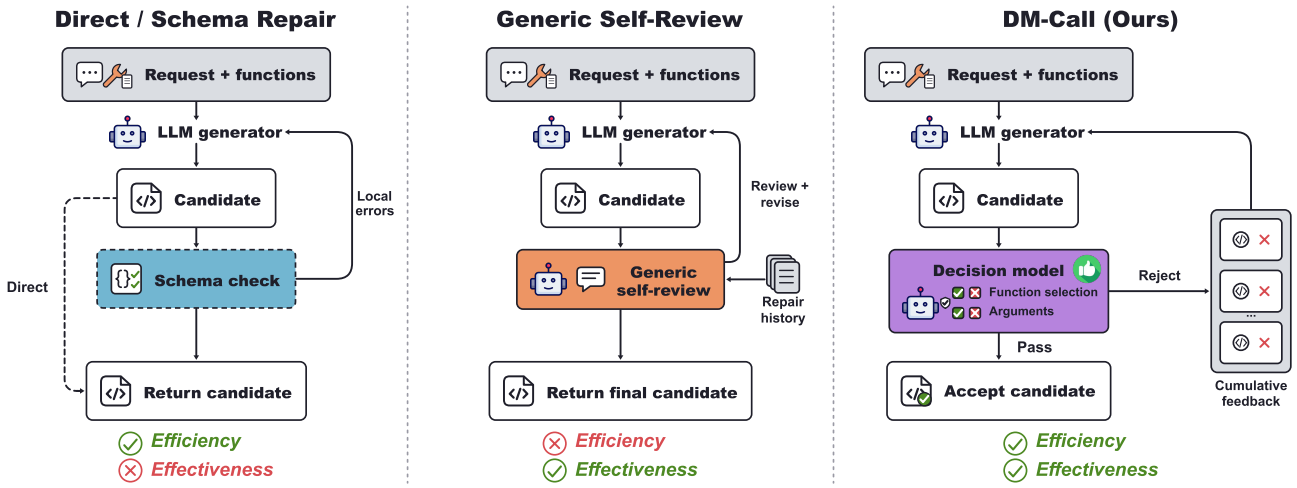


Figure 1: Comparison of function-calling strategies. DM-Call preserves an initial readable no-call before judgment. Direct generation returns the initial candidate, while schema repair revises it using local validation errors; Generic self-review refines candidates using the generator's own critiques; DM-Call combines schema validation with separate decision-model judgments of function selection and argument correctness, accumulating rejected candidates and their verification feedback to guide subsequent revisions without executing external tools.

Abstract

Reliable function calling requires Large Language Models (LLMs) to select appropriate functions and generate arguments that match user intent. Syntactically valid calls, however, can still be semantically incorrect due to wrong function selection or mismatched arguments. In this paper, we propose **Decision-Model-Guided Function Calling (DM-Call)**, an inference-time framework that combines schema validation with separate decision-model judgments of function selection and argument correctness. Building on these judgments, **DM-Call** iteratively refines candidate calls using cumulative verification feedback from previously rejected attempts without executing external tools. Extensive experiments on two benchmark datasets, including (1) BFCL single-turn cases and (2) an adapted API-Bank first-call benchmark, show that **DM-Call** improves function-calling accuracy over direct generation while offering competitive cost efficiency against refinement baselines. Ablation studies further highlight the benefits of joint verification and cumulative feedback, showcasing the impact of feedback representations and the trade-off between accuracy and inference latency across repair budgets.[1]

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**.

Keywords

function calling, decision model, evaluation, in-context learning

ACM Reference Format:

Anonymous Author(s). 2018. DM-Call: Decision-Model-Guided Function Calling with Cumulative Verification Feedback. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Function calling enables Large Language Models (LLMs) to interact with external tools and APIs by translating natural-language requests into structured function invocations [3, 10]. In detail, given a user request and available functions, an LLM determines whether a function call is needed and, if so, selects an appropriate function and generates arguments that reflect user intent. However, syntactically valid calls may still be semantically incorrect due to inappropriate function selection or mismatched arguments [8].

[1]Code is available at: github.com/gogoymh/decision-model-guided-function-calling

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Prior work has explored various approaches to improving function-calling reliability through validation and iterative refinement. Specifically, schema repair addresses structural errors, while Self-Refine revises candidate outputs using model-generated feedback [4]. Other approaches, such as multi-agent debate and Reinforced Agent, incorporate feedback from multiple agents or a separate reviewer [2, 11]. More closely related, ToolVerifier applies self-verification to tool selection and parameter generation [5].

However, several limitations remain: (1) syntactic constraints and schema validation alone cannot assess whether function selection and arguments align with user intent, (2) generic self-refinement and multi-agent or reviewer-based approaches may introduce additional inference overhead without consistently providing targeted verification, and (3) iterative refinement based only on the latest feedback may discard useful information from earlier attempts.

To address these limitations, we propose **Decision-Model-Guided Function Calling (DM-Call)**, an inference-time framework that integrates local schema validation with separate decision-model judgments of function selection and argument correctness. When a candidate fails verification, **DM-Call** retains the rejected call along with its verification scores, judgments, and schema errors to guide subsequent revisions. This cumulative feedback enables the generator to refine candidate calls based on previous failures without executing external functions. We retain the initial prediction if it is a parseable no-call or if no revision passes verification within the repair budget.

We evaluate **DM-Call** using four LLM generators [1, 6, 7] and two decision-model verifiers [9, 12] on the Berkeley Function Calling Leaderboard (BFCL) single-turn cases [8] and an adapted API-Bank first-call benchmark [3]. **DM-Call** improves function-calling accuracy over direct generation by an average of 2.89 and 0.91 percentage points on BFCL and API-Bank, respectively. Comparisons with refinement baselines further examine accuracy and inference costs. Moreover, ablation studies demonstrate the benefits of cumulative feedback while examining verification components and feedback representations, alongside accuracy–latency trade-offs across repair budgets.

Our core contributions include (1) a decision-model-guided framework that combines schema validation with separate verification of function selection and argument correctness, (2) a cumulative feedback mechanism that preserves rejected calls and their verification signals for iterative refinement, and (3) comprehensive evaluations against existing baselines, with ablation studies examining verification components, feedback strategies, and inference efficiency.

2 Methodology

2.1 Task Definition

Given a request x and the available functions $\mathcal{F}$ with their schemas, a system returns one next action: a call $c = (f, a)$ with function $f \in \mathcal{F}$ and arguments a, or no call, $c = \varnothing$. The request may include earlier dialogue turns. The action is correct when it matches the benchmark reference, that is, the expected function with acceptable argument values, or no call when none is expected.

It can be wrong in three specific ways: (1) it calls a function when no call is expected, or returns no call when one is; (2) it picks the wrong function; or (3) its arguments are wrong or missing. A wrong call can trigger an unintended external action even when it is schema-valid. Therefore, DM-Call verifies and refines candidate calls before execution, without relying on tool-execution feedback.

2.2 Decision-Model Verification

DM-Call separates an LLM generator G, which proposes candidate calls, from a decision model D, which verifies them. At iteration t, the generator produces c_t conditioned on the request, function schemas, and a history H_t of rejected candidates and their feedback. Since $H_0 = \varnothing$, the initial candidate c_0 is generated without prior feedback. The decision model D evaluates two yes/no questions about c_t and returns the probability of *yes* for each, with the superscript (t) denoting the iteration index:

$$c_t = G(H_t, x, \mathcal{F}), \qquad p_f^{(t)} = D_f(x, \mathcal{F}, c_t), \quad p_a^{(t)} = D_a(x, \mathcal{F}, c_t). \tag{1}$$

The function question D_f assesses whether the selected function matches the request or, for a no-call candidate proposed during repair, whether no call is appropriate. The argument question D_a assesses whether the required values are present, supported by the request, and assigned to the correct parameters. For a no-call candidate, argument verification is skipped and $p_a^{(t)} = \perp$. A parseable call with missing arguments, however, still receives both judgments. Passing argument verification alone does not imply correct function selection. An exception applies to the initial candidate: if c_0 is a parseable no-call, **DM-Call** returns it without verification or repair and marks it as unverified. An unparseable output is not treated as a no-call.

2.3 Acceptance and Cumulative Repair

DM-Call accepts a candidate only when it passes a local schema check and every decision-model question that applies to it. The local schema check $S(c_t)$ is true when c_t is readable, names a function in $\mathcal{F}$ and supplies every required argument with its declared type. A question that is not asked, such as the argument question on a no call or any question on an unreadable candidate, has the missing score $\perp$ (a null value). The verdict $V(p)$ is true if and only if $p \neq \perp$ and $p \geq 0.5$. Candidate c_t is accepted, written $A_t = \text{true}$, when

$$A_t = \begin{cases} S(c_t) \wedge V\big(p_f^{(t)}\big), & c_t \text{ is a no call,} \\ S(c_t) \wedge V\big(p_f^{(t)}\big) \wedge V\big(p_a^{(t)}\big), & \text{otherwise,} \end{cases} \tag{2}$$

where $\wedge$ denotes logical AND. For instance, a call with $p_f^{(t)} = 0.9$ and $p_a^{(t)} = 0.3$ is rejected on its arguments, whereas a no call needs only $p_f^{(t)} \geq 0.5$. A readable candidate that fails S is still judged by D, so its feedback carries both the schema errors and the decision model's judgments.

When c_t is rejected (A_t is false) and a repair remains ($t < r$), **DM-Call** queries the generator for a revised candidate $c_{t+1} = G(H_{t+1}, x, \mathcal{F})$ (Eq. 1). The history H_t lists the candidates rejected before step t, oldest first, each paired with its feedback d_i:

$$\begin{aligned} H_t &= \big[(c_0, d_0), \ldots, (c_{t-1}, d_{t-1})\big], \\ d_i &= \big(\Phi(p_f^{(i)}),\ \Phi(p_a^{(i)}),\ V(p_f^{(i)}),\ V(p_a^{(i)}),\ S(c_i)\big). \end{aligned} \tag{3}$$

	gpt-5-nano			gpt-5.4-mini			claude-sonnet-5.5			gemma-4-31b-it		
Method	BFCL↑	API-Bank†↑	USD/1k↓	BFCL↑	API-Bank†↑	USD/1k↓	BFCL↑	API-Bank†↑	USD/1k↓	BFCL↑	API-Bank†↑	USD/1k↓
Direct	64.12	61.32	**0.055**	81.59	80.13	**0.707**	88.25	86.97	**4.151**	86.67	81.41	–
MAD-FC [2]	60.20	29.27	0.502	79.09	**80.98**	5.438	82.38§	73.93§	31.307	39.29§	0.85§	–
PF-FC [11]	74.54	**70.09**	1.945	81.79	76.71	2.224	86.45	83.12	6.157	84.85	77.56	–
DM-Call (*Jev*)	**74.63**	64.32	0.173	**82.54**	80.13	0.917	**88.56**	**87.39**	4.609	**86.91**	**81.62**	–
DM-Call (*PPLX*)	74.24	64.53	–	82.20	80.13	–	88.46	**87.39**	–	86.88	81.41	–

Table 1: Three-run mean accuracy (%) at $r^\star = 3$ (s.d. 0.00–3.23) and USD per 1,000 BFCL cases at quoted API prices; –: excluded, as local open-weight models (Gemma, PPLX) are hard to price. Per generator: bold best, underline second. †First-call exact match, no no-call negatives. §Wrong: peer-wave calls Sonnet refused (5%/15%) or Gemma wrote as text (92%/92%).

Here Φ is the feedback representation that renders each score for the generator; the other entries are the two verdicts and the outcome of the local schema check, with its error messages when it fails. A skipped argument question therefore appears as $\Phi(\perp)$, a null score. Section 2.4 defines our default representation, Hybrid. The generator receives these records, marked as not executed, before the unchanged original request, whereas the decision model sees only the request, the schemas and the current candidate. Neither model receives the reference answer.

The loop ends at the first accepted candidate, which **DM-Call** returns; if none of $c_0, \ldots, c_r$ is accepted, it returns the initial candidate c_0. No function is executed during verification or repair. Because acceptance reflects only the decision model's judgment, an accepted candidate can still be wrong under the benchmark metric.

2.4 Hybrid Feedback

Our default representation Φ in Eq. 3, Hybrid, keeps each probability and adds a verbal label from five bands of equal width:

$$\Phi(p) = \big(p, \ell(p)\big), \qquad \ell(p) = \begin{cases} \textit{very likely incorrect}, & 0 \le p < 0.2, \\ \textit{likely incorrect}, & 0.2 \le p < 0.4, \\ \textit{uncertain}, & 0.4 \le p < 0.6, \\ \textit{likely correct}, & 0.6 \le p < 0.8, \\ \textit{very likely correct}, & 0.8 \le p \le 1. \end{cases} \tag{4}$$

For a missing score, $\ell(\perp) = \perp$, so both fields of $\Phi(\perp)$ are null. The labels change only what the generator reads; acceptance still applies V to the numeric score (Eq. 2). For example, 0.48 and 0.52 are both labeled *uncertain*, yet $V(0.48)$ is false and $V(0.52)$ is true. Section 3.3 compares Hybrid with Numeric feedback, $\Phi(p) = p$, and Text feedback, $\Phi(p) = \ell(p)$.

3 Experiment Setting & Results

3.1 Experimental Setup

Datasets and Metrics. For BFCL, we use 3,035 single-turn Python cases, including irrelevance cases where no call is expected, scored with the official abstract syntax tree (AST) metric; we exclude 16 live-relevance cases that assess only whether a function is called. For API-Bank [3], we use 156 first-call test cases with a fixed catalog of 53 APIs, scored by exact match of function names and type-normalized arguments without executing any API.

Models. The generators are (i) `gpt-5-nano`, (ii) `gpt-5.4-mini`, (iii) `claude-sonnet-5.5` and (iv) `gemma-4-31b-it` (local vLLM, FP8), all with default sampling, at most 4,096 output tokens and native, non-parallel tool calls (GPT reasoning effort `none`; no thinking mode). The decision models are the hosted *Jev* (`jev-1.13.0`) and the open-weight 27B PPLX Decider v1.1 [9], run locally in BF16 with its calibration temperature (1.009).

Baselines. (i) Direct returns the generator's initial prediction; *(ii) Schema Repair* revises invalid calls using local schema errors; *(iii) Self-Refine* [4] revises with the generator's own critique; *(iv) MAD-FC* [2] lets three agents debate for one round and takes a majority vote, without the Direct prediction; and *(v) PF-FC* [11] has `gpt-5.4-mini` review each Direct prediction and refine it up to three times. As (ii) and (iii) use only local checks or self-critique, the ablation (Section 3.3) compares them with DM-Call variants.

Implementation. A decision-model question passes at $p \ge 0.5$ (Eq. 2); DM-Call uses Hybrid feedback (Eq. 4) and at most $r^\star = 3$ repairs, i.e., up to four generator calls per case (Section 3.3). Direct, PF-FC and DM-Call share the first answers c_0 in each of three runs.

3.2 Main Results

On BFCL, DM-Call with *Jev* is the most accurate for every generator (Table 1), improving on Direct by 10.5 points for `gpt-5-nano`, 0.9 for `gpt-5.4-mini` and 0.2–0.3 for the two strongest; *PPLX* stays within 0.4 points of *Jev*. API-Bank gains are smaller (0–3.2 points), and PF-FC and MAD-FC lead there with `gpt-5-nano` and `gpt-5.4-mini`. Each extra correct BFCL call over Direct costs \$0.0011 (`gpt-5-nano`) and \$0.022 (`gpt-5.4-mini`); PF-FC and MAD-FC cost 1.3–11 and 3–7 times as much as DM-Call.

3.3 Ablation

Verification and history. In Table 3, Schema Repair and Self-Refine gain 2.1 and 5.8 points over Direct and the decision-model variants 8.9 to 10.2, so external judgments catch errors the generator misses. Both checks together give the best call accuracy and argument-only feedback (**DM-Call-A**) the best argument accuracy; latest-only feedback (**DM-Call-L**) loses 1.3 points, suggesting that earlier rejected calls keep the generator from repeating them.

Feedback representation. Table 2 varies only how scores reach the generator: probabilities and thresholds (*Numeric*), the labels of Eq. 4 (*Text*), or both (*Hybrid*). Hybrid is clearly better only for `gpt-5-nano` on BFCL (+0.6 points over Numeric), which repairs most, and never clearly worse, so it stays the default.

Repair budget. Extended to $r = 10$ (Figure 2), accuracy gains only 1.8 points from $r = 3$ to 10 while p95 time rises from 9.7 to

Generator	Numeric (%)	Text (%)	Hybrid (%)
BFCL (3,035 cases/run)			
`gpt-5-nano`	73.72 ± 0.17	73.86 ± 0.18	**74.32** ± 0.22
`gpt-5.4-mini`	82.14 ± 0.10	**82.20** ± 0.08	82.11 ± 0.11
`claude-sonnet-5.5`	88.58 ± 0.30	**88.61** ± 0.31	88.58 ± 0.28
`gemma-4-31b-it`*	**86.93** ± 0.10	86.91 ± 0.11	86.91 ± 0.12
API-Bank (156 cases/run)			
`gpt-5-nano`	**61.97** ± 2.67	61.11 ± 2.06	61.54 ± 2.56
`gpt-5.4-mini`	**78.63** ± 0.37	**78.63** ± 0.37	**78.63** ± 0.37
`claude-sonnet-5.5`	**87.18** ± 0.00	**87.18** ± 0.00	**87.18** ± 0.00
`gemma-4-31b-it`*	**81.62** ± 0.37	**81.62** ± 0.37	**81.62** ± 0.37

Table 2: Representations at $r^\star = 3$: three-run mean ± s.d.; archived initials (*Table 1's); bold: row max.

Method	Call (%)	Func. (%)	Args. (%)
Direct	64.11 ± 0.31	75.11 ± 0.29	81.27 ± 0.14
Schema repair	66.25 ± 0.42	76.96 ± 0.46	81.76 ± 0.20
Self-refine	69.94 ± 0.30	80.21 ± 0.42	82.07 ± 0.15
DM-Call-F	73.12 ± 0.46	83.84 ± 0.52	81.78 ± 0.07
DM-Call-A	73.61 ± 0.33	83.73 ± 0.52	**82.64** ± 0.38
DM-Call-L	73.05 ± 0.24	83.61 ± 0.22	82.05 ± 0.16
DM-Call	**74.32** ± 0.22	**84.80** ± 0.26	82.18 ± 0.13

Table 3: Archived replay, `gpt-5-nano`/BFCL, *Jev*, $r = 3$: three-run mean ± s.d.; Args.: given correct functions.

24.3 s. We place $r^\star$ at the knee, $r^\star = \arg\max_{r \in R}(\widetilde{\text{Acc}}_r - \widetilde{C}_r)$ over $R = \{1, 2, 3, 5, 7, 10\}$, where $\widetilde{X}_r = (X_r - X_1)/(X_{10} - X_1)$ rescales mean accuracy Acc_r and requests per case C_r. This gives $r^\star = 3$; $r = 5$ would add 0.9 points for 16% more requests and 42% higher p95, and one of the three runs alone favors $r = 5$.

3.4 Discussion

DM-Call helps most when the generator errs often and stays cheap because 76% (`gpt-5-nano`) and 94% (`gpt-5.4-mini`) of BFCL cases need no repair. Part of the baselines' loss comes from unusable outputs, scored as wrong: Sonnet refused 5%/15% of MAD-FC peer-wave requests (BFCL/API-Bank), Gemma wrote 92% of its peer-wave calls as text on both benchmarks, and PF-FC's reviewer was unreadable for 2–5% of cases, explaining 174 of the 205 correct Sonnet first calls it lost on BFCL. *Limitations:* thresholds, fallback, label cuts and $r^\star$ were chosen on these benchmarks without a held-out split; inputs are mostly English, BFCL has ten known annotation errors, and our API-Bank adaptation covers only first calls.

4 Conclusion

DM-Call checks function choice and arguments with a decision model before execution and repairs rejected calls with cumulative Hybrid feedback. It helps most with weak generators at little cost; held-out and execution-based evaluation remain open.

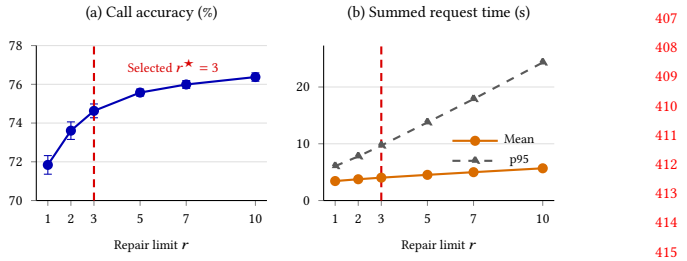


Figure 2: DM-Call, `gpt-5-nano`/BFCL, *Jev*: three $r = 10$ runs cut at each r (bars: s.d.; dashed: $r^\star = 3$).

References

[1] Anthropic. 2026. *System Card: Claude Sonnet 5.5.* Technical Report. Anthropic. https://www.anthropic.com/claude-sonnet-5-5-system-card

[2] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235).* PMLR, 11733–11763. https://proceedings.mlr.press/v235/du24e.html

[3] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing.* 3102–3116. doi:10.18653/v1/2023.emnlp-main.187

[4] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. 46534–46594. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[5] Dheeraj Mekala, Jason Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. 2024. ToolVerifier: Generalization to New Tools via Self-Verification. *arXiv preprint arXiv:2402.14158* (2024). https://arxiv.org/abs/2402.14158

[6] OpenAI. 2025. GPT-5 Nano Model. OpenAI API Documentation. Model snapshot: gpt-5-nano-2025-08-07. https://developers.openai.com/api/docs/models/gpt-5-nano

[7] OpenAI. 2026. GPT-5.4 Mini Model. OpenAI API Documentation. Model snapshot: gpt-5.4-mini-2026-03-17. https://developers.openai.com/api/docs/models/gpt-5.4-mini

[8] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267).* PMLR, 48371–48392. https://proceedings.mlr.press/v267/patil25a.html

[9] Perplexity AI. 2026. pplx-decider-v1.1-27b. Hugging Face Model Card. https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b

[10] Timo Schick, Jane Dwivedi-Yu, Roberto Dessí, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. 68539–68551. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[11] Anh Ta, Junjie Zhu, and Shahin Shayandeh. 2026. Reinforced Agent: Inference-Time Feedback for Tool-Calling Agents. In *Proceedings of the Fifth Workshop on Generation, Evaluation and Metrics (GEM).* Association for Computational Linguistics, San Diego, California, USA, 136–147. doi:10.18653/v1/2026.gem-main.13

[12] TypeSafe. 2026. Models: Jev and the System One API. Technical documentation. Accessed October 15, 2026. https://docs.typesafe.ai/models